

Project 2

(Due: Oct 12, 2026)

The project will be done in pairs.

In this project, you are to make two changes to Project 1 to solve the same problem, namely the 4 by 4 sliding tile puzzle: (a) replace the tree search by graph search and (b) replace the unguided search by informed search. The pseudo-code for graph-based best-first search is given below:

```

Procedure A*
Input: Implicit problem graph with start node  $s$ , weight function  $w$ , heuristic  $h$ ,
        successor generation function  $Expand$ , and predicate  $Goal$ 
Output: Cost-optimal path from  $s$  to  $t \in T$ , or  $\emptyset$  if no such path exists

Closed  $\leftarrow \emptyset$                                 ;; Initialize structures
Open  $\leftarrow \{s\}$                                 ;; Insert  $s$  into search frontier
 $f(s) \leftarrow h(s)$                                 ;; Initialize estimate
while (Open  $\neq \emptyset$ )                                ;; As long as there are frontier nodes
    Remove  $u$  from Open with minimum  $f(u)$             ;; Select node for expansion
    Insert  $u$  into Closed                                ;; Update list of expanded nodes
    if ( $Goal(u)$ ) return Path( $u$ )                        ;; Goal found, return solution
    else  $Succ(u) \leftarrow Expand(u)$                     ;; Expansion yields successor set
        for each  $v$  in  $Succ(u)$                             ;; For all successors  $v$  of  $u$ 
             $Improve(u, v)$                                 ;; Call relaxation subroutine
return  $\emptyset$                                 ;; No solution exists

Procedure Improve
Input: Nodes  $u$  and  $v$ ,  $v$  successor of  $u$ 
Side effects: Update parent of  $v$ ,  $f(v)$ , Open, and Closed

if  $v$  in Open                                ;; Node already generated but not expanded
    if ( $g(u) + w(u, v) < g(v)$ )                    ;; New path is cheaper
         $parent(v) \leftarrow u$                         ;; Set predecessor pointer
         $f(v) \leftarrow g(u) + w(u, v) + h(v)$             ;; DecreaseKey operation
    else if  $v$  in Closed                            ;; Node  $v$  already expanded
        if ( $g(u) + w(u, v) < g(v)$ )                    ;; New path cheaper
             $parent(v) \leftarrow u$                         ;; Set predecessor pointer
             $f(v) \leftarrow g(u) + w(u, v) + h(v)$             ;; Update estimate
            Remove  $v$  from Closed                        ;; Reopening of  $v$ 
            Insert  $v$  into Open with  $f(v)$                 ;; Reopening of node
    else                                            ;; Node not seen before
         $parent(v) \leftarrow u$                         ;; Set predecessor pointer
        Initialize  $f(v) \leftarrow g(u) + w(u, v) + h(v)$     ;; First estimate
        Insert  $v$  into Open with  $f(v)$                     ;; Add  $v$  to search frontier

Algorithm 2.13
A*.

```

The problem we shall solve involves solving a widely known puzzle called the sliding tile puzzle, shown below. The puzzle involves n^2 tiles in a square board numbered 1 to $n^2 - 1$ with an empty square to which one can slide an adjacent tile in U, D, L or R sliding move.



Given a starting state s and a goal state g , your program will find the shortest move sequence to reach g from s , and print the move sequence. It should also output the total number of nodes expanded (removed from the queue), the maximum size of the queue and the CPU time taken. The methods to be implemented are BFS, DLS and IDS. In each case, we are using tree search approach which means a state may enter the queue or stack even if it has already entered before. The only reentry we will avoid is the entry of a parent node as a successor of a child node.

As we have noted, DFS may fail to find the best solution, and it may loop forever since in the tree search version. So instead of DFS, we will implement DLS, the depth-limited version of DFS which is essentially DFS performed to specified depth L . Thus, DLS takes as input a start state, a goal state and a depth L . It will search for a solution to depth L (the root is considered to be at depth 0) and report a solution if it exists, else returns failure. DLS forms the basis for IDS which calls DLS for depth set to 0, 1, 2, until a solution is found.

Your choice of data structures should be to make the algorithm perform efficiently but it should also be simple so that testing and debugging can be done effectively and in a reasonable time frame.

You are to implement the following heuristics:

Heuristic 1: Sum of the distances of the tiles to the goal (also known as the Manhattan distance heuristic).

Heuristic 2: To define the function h_2 , we replace the tiles numbered 1 to 8 by a Red tile and the tiles numbered 9 to 15 by a Green tile. This maps each state of the puzzle to a state with 8 Red tiles, 7 Green tiles and an empty slot. The state space of this simplified board has size $16 \times \binom{15}{8} = 102,960$ which is significantly smaller than $\frac{16!}{2}$. Let G be state space graph of the set of boards with Red and Green tiles. Let g' be the state in G to which the goal state g is mapped. For a state s in the original state, let s' be its mapped state in G .

$h_2(s)$ = the distance between s' and g' in G . $h_2(s)$ is computed as follows:

IDS is performed from g' to each state s' in G and is stored in a table. $h_2(s)$ is read off from the table entry for s' which is the state to which s is mapped under the red, green color mapping. There is an overhead associated with computing $h_2(s)$ but this is a one-time cost.

Heuristic h_3 : $h_3(s)$ is chosen as: $\max\{h_1(s), h_2(s)\}$.

Input/output format: The input will be represented as a vector of length 4 each member in turn a vector of length 4. 0 will be used for the empty slot. For example, the above board will be represented as

[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 0]].

Your UI should be such that it asks the user to enter the start state in the above format, then it asks for the goal state (again in the same format). Then it asks the user to choose the algorithm to run: (1) Best First Search with heuristic 1 (2) Best First Search with heuristic 2 (3) Best First Search with heuristic 3 and (4) halt. The program will loop through the options until option 5 is chosen. Please allow the user to enter any combination of {1, 2, 3} such as 123 etc.

Output format will be:

The solution found (or not found).

The number of moves: _____

The sequence of moves: (as a string over the alphabet {L, R, U, D})

The number of states removed: _____ (the number of states removed from OPEN set)

The maximum size of the queue (stack): _____

CPU time: _____

Some test cases will be provided soon.

What should be submitted? In addition to the source code and compiled executable, please include the following:

- Design documentation:

Please describe the overall structure of your code. Describe the choice of data structures and implementation details for the main functions (such as best-first-search and IDS used for computing h_2).

- Code documentation:

List all the functions you have implemented. For each function, state exactly the inputs and outputs. Also state pre-conditions on the input.

- Instructions for compiling and running the code. Create an input pair (s, g) such that requires at least 30 moves to reach g from 2. Run your program with input (s, g) and with option 123.

- State clearly describe the use of AI. This requires (a) summary of what parts of coding was done using AI and (b) the full transcript of the dialog with AI assistant.

- Role of each person in the team.